

FotoMac Lab — a bancada está pronta para as 300 fotos

O que mudou, a página visual e como testar

Thiago José Nunes

23 de setembro de 2026 (atualizado)

1. A resposta curta

Não estava pronto. Agora está.

O que existia era o `fotomac_face.py`: um motor que compara **duas** imagens e imprime o resultado no terminal. Para o teste que você quer — varrer um acervo inteiro, contar correspondências, listar os arquivos e cronometrar — faltava tudo: não havia varredura de pasta, não havia medição de tempo, não havia arquivo de saída, e uma foto quebrada no meio derrubaria a execução inteira.

Criei o `benchmark.py`, que é a peça que faltava. Ele foi testado com **302 arquivos** (as suas 6 fotos replicadas até virar um acervo do tamanho real, mais um arquivo corrompido e um `.heic` falso de propósito).

2. O que mudou

2.1 `claude.md` — reescrito

Deixou de ser um parágrafo corrido e virou um documento com sete seções: identidade do projeto, o que o teste precisa responder, estado dos dados, estrutura, comandos, convenções e armadilhas da medição.

A parte que mais muda o comportamento do agente são duas frases no topo: *"isto não é o produto, é a bancada"* e *"o modelo é peça trocável"*. Elas evitam que o código cresça como se fosse o sistema final.

2.2 `benchmark.py` — novo

O programa que roda o teste. Em uma passada ele:

1. carrega os modelos e cronometra esse custo;
2. varre a pasta do acervo, detecta rostos e gera um vetor por rosto, cronometrando;
3. pega a foto de consulta (o autorretrato) e a compara com o acervo inteiro;
4. imprime o relatório e grava os arquivos de resultado.

O tempo vem separado em três — e isso é de propósito

Fase	O que é	Medido com 302 fotos
Carregar modelos	custo único, ao iniciar	0,03 s
Indexar o acervo	detectar + gerar vetor de cada foto	23,7 s (12,7 fotos/s)
Buscar	comparar a consulta com o acervo já indexado	0,52 ms

Você tinha pedido um número só: "quanto tempo para passar pelas 300 fotos". Somados dá ~24 segundos — mas esse número engana. No sistema real a indexação roda **uma vez**, de madrugada, sem ninguém esperando. A busca roda com o **visitante parado na frente do totem**. São 24 segundos contra meio milésimo de segundo. Se você somar os dois, o relatório do TCC vai dizer que o sistema demora 24 segundos para responder — e não demora.

Medição feita num Apple M5 Pro, com MAX_SIDE=1920. Tempo sem máquina e sem resolução registradas não compara com nada, então o relatório grava as duas coisas.

Nada derruba a varredura

Cada foto termina em uma de três categorias, e todas aparecem no relatório:

Categoria	Significa
ok	rosto detectado e vetor gerado
sem_rosto	o detector não achou rosto (foto de paisagem, rosto de perfil, muito pequeno)
ilegível	o OpenCV não abriu o arquivo — tipicamente HEIC do iPhone

Isso importa de verdade: se o fotógrafo mandar HEIC em vez de JPEG, você vai ver "150 ilegíveis" no relatório em vez de um programa que trava ou, pior, um resultado silenciosamente pela metade.

Varredura de limiares

O relatório não usa só o limiar 0.363. Ele mostra quantas fotos corresponderiam em onze limiares diferentes, de 0.20 a 0.70. Motivo: se você fixar um número e contar, você mediu o **número que escolheu**, não o modelo. A tabela mostra onde a contagem muda — é ali que o limiar se calibra.

Os arquivos de saída

Gravados em resultados/, com data e hora no nome:

- <data>_acervo.csv — uma linha por foto: estado, quantos rostos, quanto tempo levou.
- <data>_consulta_<nome>.csv — **todas** as fotos com a similaridade de cada uma, não só as que passaram do limiar. Assim você recalcula qualquer limiar depois, no Excel, sem reindexar as 300 fotos.
- <data>_resumo.json — tudo junto, para virar gráfico.

3. Como testar

3.1 O teste rápido, agora, com as 6 fotos

Abra o terminal na pasta do projeto e rode:

```
cd ~/fotomac-lab  
.venv/bin/python benchmark.py --consulta fotos/IMG_1128.JPG
```

Use sempre `.venv/bin/python`, nunca `python` sozinho — o OpenCV só está instalado dentro do `.venv`.

O que você deve ver: 6 fotos encontradas, 7 rostos indexados, e três correspondências — IMG_1128, IMG_1129 e IMG_1130, com similaridades 1.0000, 0.5316 e 0.5202. Um aviso vai dizer que a foto de consulta está dentro do acervo e por isso casa consigo mesma com 1.0000. Ao final, os caminhos dos arquivos gerados.

Depois abra o CSV e confira com os seus olhos:

```
open resultados/
```

3.2 Testar com outro limiar

```
.venv/bin/python benchmark.py --consulta fotos/IMG_1128.JPG --limiar 0.55
```

Com 0.55 a contagem cai de 3 para 1. É o mesmo modelo e o mesmo acervo — só a régua mudou. É exatamente esse efeito que a tabela de limiares existe para mostrar.

3.3 Testar várias consultas de uma vez

```
.venv/bin/python benchmark.py \  
--consulta fotos/IMG_1128.JPG \  
--consulta fotos/man-with-his-arms-crossed.jpg
```

O acervo é indexado **uma vez só** e as duas buscas rodam em cima do mesmo índice. Com 300 fotos isso economiza 24 segundos por consulta extra.

3.4 Provar que arquivo quebrado não derruba nada

```
mkdir -p /tmp/teste-fotomac  
cp fotos/IMG_1128.JPG fotos/IMG_1129.JPG /tmp/teste-fotomac/  
echo "isto nao e uma imagem" > /tmp/teste-fotomac/quebrada.heic  
.venv/bin/python benchmark.py --acervo /tmp/teste-fotomac --consulta fotos/  
IMG_1130.JPG
```

O que você deve ver: 3 fotos encontradas, 2 com rosto, 1 ilegível — e o programa terminando normalmente, com o arquivo problemático nomeado no CSV.

4. A página visual

Uma tabela diz *quantas*. Ela não diz *quem*. Com 204 fotos de formatura, onde uma única foto tem até 21 rostos, "similaridade 0,3373" é um número sem prova — você precisa ver de qual rosto ele saiu.

A página é gerada junto com o CSV, a cada execução:

resultados/<data-hora>_relatorio.html

Abre com dois cliques, direto do disco. **Não é um site e não vai para a internet** — as fotos do fotógrafo são privadas, então a página fica local e referencia os arquivos por caminho relativo. Nada sai da sua máquina.

O que ela mostra

- **Todas as 197 fotos com rosto**, em grade, ordenadas da mais parecida para a menos.
- Em cada miniatura, **caixa verde** no rosto que gerou a similaridade e **caixas cinzas** nos outros rostos da mesma foto.
- No canto de cada miniatura, um **recorte ampliado do rosto que casou**. Esta é a peça que faz a página valer: numa foto com 20 pessoas, a caixa do rosto responsável tem uns 15 pixels e some. O recorte mostra quem o modelo realmente comparou.
- Um **controle de limiar** que arrasta de 0 a 1 e repinta a grade na hora, sem reindexar nada. As similaridades já estão na página; o limiar só decide onde cai a régua.
- Fotos até 0,05 abaixo do limiar ficam marcadas como **limítrofes**, em laranja — são as que entrariam se você afrouxasse um pouco, e as que valem olhar uma a uma.
- Clique em qualquer miniatura e o **original abre em tamanho cheio**.

O que ela já revelou

A foto IMG_3008 deu similaridade **0,3373** com o autorretrato — logo abaixo do limiar de 0,363. Na tabela, isso é só um número quase passando. Na página, o recorte mostra que o rosto responsável era o de uma **mulher desfocada ao fundo do salão**. Pessoa completamente diferente.

Atualização de 23/09, mais tarde no mesmo dia: com o gabarito em mãos, o limiar foi recalibrado de 0,363 para **0,45**. O 0,363 padrão do SFace produzia um falso positivo real (IMG_3011, 0,3728). Ver SISTEMA-DE-PONTUACAO.md — ele substitui a seção 6.1 deste documento.

5. O acervo real — o que medimos

As 204 fotos (6 de amostra + ~200 do fotógrafo, 957 MB, todas JPG):

Medida	Resultado
Fotos no acervo	204
Fotos com rosto detectado	197
Fotos sem rosto	7
Arquivos ilegíveis	0 — nenhum HEIC, o risco não se concretizou
Rostos indexados	1004
Carregar modelos	0,05 s
Indexar o acervo	22,5 s (9,1 fotos/s)
Buscar	4,1 ms
Gerar as miniaturas	+6 s

Mil rostos em 197 fotos: são fotos de plateia, com muita gente ao fundo. Isso muda a natureza do teste — não é "uma pessoa por foto".

A varredura de limiares no acervo real

Limiar	Fotos correspondentes
0,20	83
0,25	30
0,30	12
0,363 (padrão SFace)	3
0,50	3
0,55	1

O salto entre 0,30 e 0,363 é onde o lixo de fundo entra. Foi olhando as limítrofes na página que ficou claro que esse lixo é lixo mesmo.

6. O que ficou de fora — e por quê

6.1 O gabarito existe agora — e mudou a conclusão

Quando este documento foi escrito, não havia gabarito, e por isso ele dizia que o teste media desempenho e não precisão. **Isso foi resolvido no mesmo dia:** as fotos IMG_1128, IMG_1129 e IMG_1130 são as únicas com o Thiago, e nenhuma das ~200 fotos do fotógrafo tem ele. Isso é um gabarito completo para uma pessoa.

Com ele foi possível medir precisão de verdade, descobrir que o limiar padrão errava, e recalibrá-lo. O sistema de pontuação está documentado em **SISTEMA-DE-PONTUACAO.md**, que é o documento a ser usado daqui para frente.

Fica a ressalva que continua valendo: o gabarito cobre **uma pessoa**. Ampliá-lo para três ou quatro é o próximo passo que mais fortalece o TCC.

6.2 Outras coisas que não fiz, de propósito

- **Não indexei em paralelo.** Daria para cair de 24 s para uns 7 s usando os vários núcleos, mas isso distorce a medição de tempo por foto, que é o número que você precisa para comparar modelos. Velocidade bruta é otimização; agora você quer medida limpa.
- **Não guardei os vetores em disco.** Reindexar 300 fotos custa 24 segundos; um cache economizaria isso ao custo de mais uma peça para dar errado. Se o acervo crescer para milhares, aí vale.
- **Não toquei no `fotomac_face.py`.** Ele funciona e está bem escrito. A bancada é uma camada por cima, não uma reescrita — é o que permite trocar o modelo depois sem mexer no resto.
- **Não instalei nenhuma dependência nova.** Só OpenCV e NumPy, que já estavam lá.